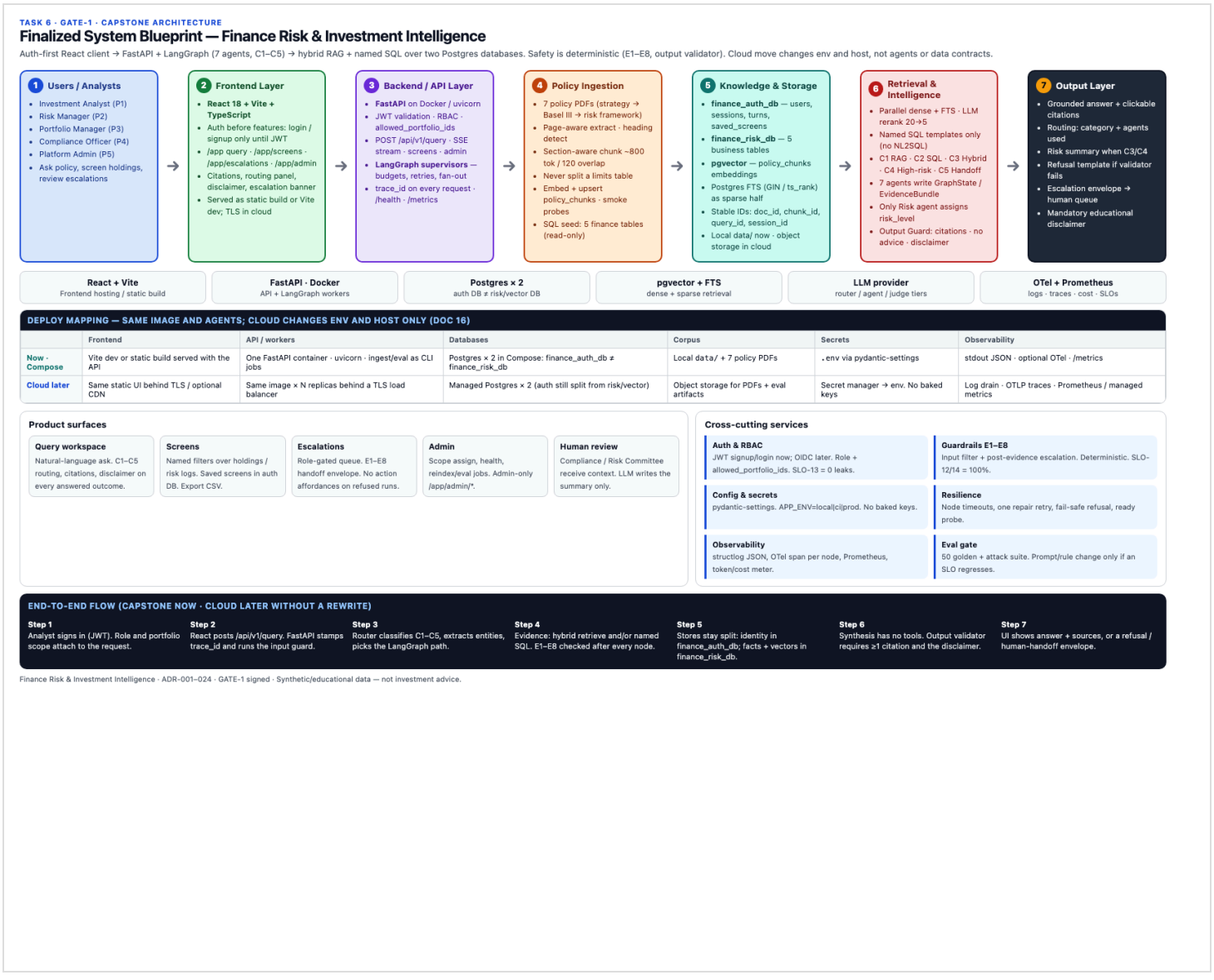
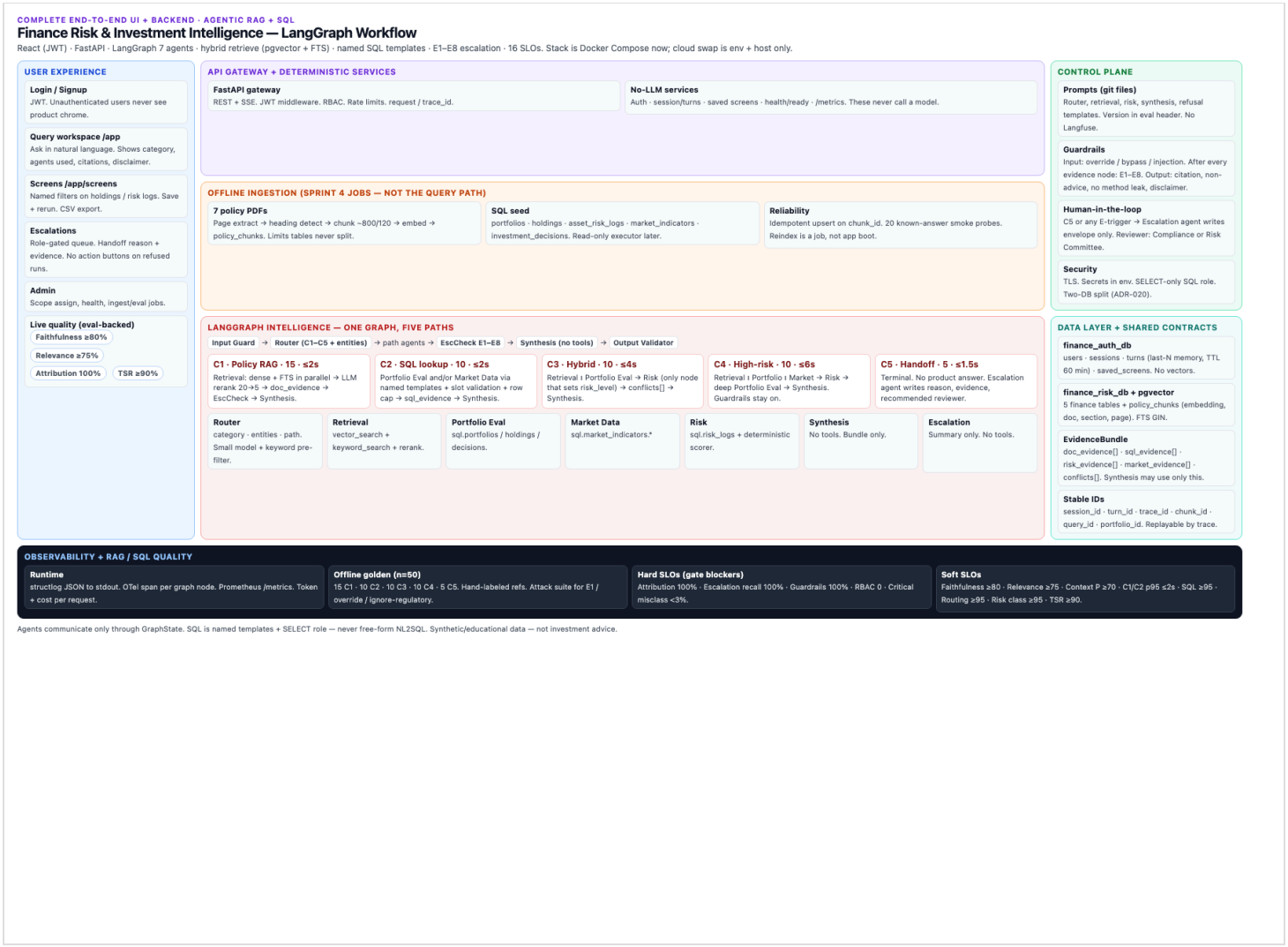
06 — High Level Architecture

Task 6 — Sprint 2 — Foundation & Planning

Submission diagrams (lead): - Architecture — deliverables/diagrams/System_Blueprint.html · PNG png/09_system_blueprint.png - Workflow — deliverables/diagrams/Agentic_RAG_SQL_Workflow.html · PNG png/10_agentic_rag_sql_workflow.png - Two-tab viewer — deliverables/diagrams/Task6_Architecture_and_Workflow.html





Older Excalidraw / C4 HTML stays in `deliverables/diagrams/` as backup. Narrative plus mermaid below is the binding detail.

---

1. Architectural Narrative

The system is a **FastAPI service fronting a LangGraph state machine** that reasons over two evidence sources: a pgvector index of 7 financial policy PDFs, and the read-only `finance_risk_db` relational store of portfolios, holdings, risk logs, market indicators, and investment decisions.

A request travels through four bands:

Band 1 — Boundary. FastAPI authenticates the caller, resolves `role` and `allowed_portfolio_ids`, and stamps a `trace_id`. An input guardrail runs *before* any model call and classifies override / bypass / injection intent. Nothing expensive happens until the request is known to be legitimate.

Band 2 — Understanding. A cheap, fast model plus a deterministic keyword pre-filter classifies the query into one of five categories (C1 RAG, C2 SQL, C3 Hybrid, C4 High-Risk, C5 Escalation), assigns a preliminary risk level, and extracts entities (portfolio, asset, sector, region, indicator, threshold, date range). Category

determines the path; path determines cost and latency. This is the single most important accuracy lever in the system, which is why Query Routing Accuracy has its own 95% SLO.

Band 3 — Execution. LangGraph runs the path. Simple categories take one hop. Hybrid and High-Risk categories fan out to Retrieval, Portfolio Evaluation, and Market Data **in parallel** (they have no data dependency), then fan in to the Risk agent, which is the only component permitted to assign a risk level. All agents write typed evidence into a shared `EvidenceBundle`; none of them talk to each other directly. Deterministic escalation rules (E1–E8) are evaluated after every evidence-producing node, so a `escalation_flag = TRUE` row or a policy-limit breach diverts the run to the Escalation terminal node regardless of how far the reasoning has progressed.

Band 4 — Synthesis and validation. The Synthesis agent has **no tools**. It can only compose from the evidence bundle, which is what makes Faithfulness measurable and Source Attribution Rate enforceable at 1.0. An output validator then checks: every claim cited, no investment advice, no leaked override method, disclaimer present. Failing validation replaces the answer with a refusal template rather than shipping an uncited answer.

Cross-cutting, every node emits an OpenTelemetry span and a structured JSON log line with token and cost accounting, so any answer can be reconstructed and explained after the fact — the auditability requirement that a financial-governance context demands.

Why this shape

Force	Architectural response
16 SLOs including four binary safety gates	Deterministic rule engine for safety; LLM only where judgement is genuinely needed
SQL Correctness >=95%	Named parameterized templates; no free-form text-to-SQL
Latency <=2s on common paths	Category-conditional depth: one hop for C1/C2, fan-out only for C3/C4
Source Attribution Rate = 1.0	Tool-less synthesis over a typed evidence bundle + post-hoc citation validator
Escalation Recall = 1.0	Escalation is a deterministic pre-emptive graph transition, not a prompt instruction
Auditability	Span per node, evidence bundle persisted, replayable by `trace_id`
Small team, 4-week window	One database for both vector and relational; boring stack; no service mesh

---

2. System Context (C4 Level 1)

```
graph TB
    subgraph Users
        A1["Investment Analyst<br/>(P1)"]
        A2["Risk Manager<br/>(P2)"]
        A3["Portfolio Manager<br/>(P3)"]
        A4["Compliance Officer<br/>(P4)"]
        A5["Platform Admin<br/>(P5)"]
    end

    SYS["<b>Finance Risk &amp;amp; Investment<br/>Intelligence System</b><br/>Multi-
```

```
agent RAG + SQL<br/>with guardrails &amp;amp; escalation"]

    subgraph Evidence Sources
        D1[("Policy Corpus<br/>7 PDFs<br/>strategy — risk — regulatory —<br/>market — monitoring —<br/>Basel III — risk framework")]
        D2[("finance_risk_db<br/>portfolios — portfolio_holdings —<br/>asset_risk_logs — market_indicators —<br/>investment_decisions")]
    end

    LLM["OpenAI-compatible LLM<br/>chat + embeddings"]
    HUM["Human Analyst Queue<br/>(escalation handoff)"]
    OBS["Observability<br/>logs — traces — metrics — cost"]

    A1 & A2 & A3 & A4 --> SYS
    A5 -.admin.-> SYS
    SYS -->|semantic retrieval| D1
    SYS -->|read-only parameterized SQL| D2
    SYS <-->|inference| LLM
    SYS ==>|mandatory escalation<br/>with full context| HUM
    SYS --> OBS
```

---

3. Container / Layer View (C4 Level 2)

```
graph TB
    CLIENT["Frontend / API client<br/>(Sprint 5+)"]

    subgraph L1["L1 — Query Interface — FastAPI"]
        API["POST /api/v1/query"]
        AUTH["Auth + RBAC scope<br/>role, allowed_portfolio_ids"]
        HLTH["/health — /metrics — /admin/*"]
    end

    subgraph X1["X1 — Safety &amp;amp; Governance (cross-cutting)"]
        GIN["Input Guardrail<br/>override — bypass — injection"]
        ESC["Escalation Rule Engine<br/>E1–E8 deterministic"]
        GOUT["Output Validator<br/>citations — non-advice — no method leak"]
    end

    subgraph L2["L2 — Query Understanding"]
        RT["Router Agent<br/>category — risk_level — entities — path"]
    end

    subgraph L3["L3 — Agent Orchestration — LangGraph"]
        GRAPH["State machine<br/>GraphState + conditional edges<br/>node timeouts + retries"]
    end
```

```
    subgraph L4["L4 — Knowledge Retrieval"]
        RETR["Retrieval Agent"]
        HYB["Hybrid retriever<br/>vector + BM25 + rerank"]
        VS[("pgvector<br/>policy_chunks")]
    end

    subgraph L5["L5 — Data Analysis"]
        PEV["Portfolio Evaluation Agent"]
        MDA["Market Data Agent"]
        RSK["Risk Agent<br/>+ deterministic scorer"]
        TPL["SQL Template Registry<br/>named — parameterized"]
        EXEC["Executor (read-only role)<br/>slot validation — row cap — audit"]
        DB[("finance_risk_db")]
    end

    subgraph L6["L6 — Response Synthesis"]
        SYN["Synthesis Agent<br/>NO TOOLS"]
        CIT["Citation Formatter"]
    end

    subgraph X2["X2 — Observability &amp;amp; Cost"]
        LOG["structlog JSON"]
        TRC["OpenTelemetry spans"]
        MET["Prometheus metrics"]
        CST["Token / cost meter"]
    end

    MEM[("Session Memory<br/>last N turns + entities")]

    CLIENT --> API --> AUTH --> GIN --> RT --> GRAPH
    GRAPH --> RETR --> HYB --> VS
    GRAPH --> PEV --> TPL
    GRAPH --> MDA --> TPL
    GRAPH --> RSK --> TPL
    TPL --> EXEC --> DB
    GRAPH --> ESC
    GRAPH --> SYN --> CIT --> GOUT --> API
    ESC ==>|terminal| GOUT
    GRAPH <--> MEM
    L3 -.-> X2
    L4 -.-> X2
    L5 -.-> X2
    L6 -.-> X2
```

---

4. Agent Orchestration (LangGraph)

```
stateDiagram-v2
    [*] --> InputGuardrail

    InputGuardrail --> Escalation : override / bypass / injection intent (E1)
    InputGuardrail --> Router : clean

    Router --> RAGPath : C1 Policy Interpretation
    Router --> SQLPath : C2 Data Lookup
    Router --> HybridFanout : C3 Hybrid Risk Evaluation
    Router --> HighRiskFanout : C4 High-Risk Decision
    Router --> Escalation : C5 Escalation Scenario

    state RAGPath {
        [*] --> Retrieval
        Retrieval --> [*]
    }

    state SQLPath {
        [*] --> PortfolioEval
        PortfolioEval --> [*]
    }

    state HybridFanout {
        [*] --> fork_h
        state fork_h <<fork>>
        fork_h --> Retrieval_H
        fork_h --> PortfolioEval_H
        Retrieval_H --> join_h
        PortfolioEval_H --> join_h
        state join_h <<join>>
        join_h --> RiskAgent_H
        RiskAgent_H --> [*]
    }

    state HighRiskFanout {
        [*] --> fork_x
        state fork_x <<fork>>
        fork_x --> Retrieval_X
        fork_x --> PortfolioEval_X
        fork_x --> MarketData_X
        Retrieval_X --> join_x
        PortfolioEval_X --> join_x
        MarketData_X --> join_x
        state join_x <<join>>
        join_x --> RiskAgent_X
        RiskAgent_X --> PortfolioEvalDeep
        PortfolioEvalDeep --> [*]
    }
```

```
    RAGPath --> EscalationCheck
    SQLPath --> EscalationCheck
    HybridFanout --> EscalationCheck
    HighRiskFanout --> EscalationCheck

    EscalationCheck --> Escalation : E2–E8 fired
    EscalationCheck --> Synthesis : clear
    EscalationCheck --> InsufficientEvidence : no / weak evidence (E5)

    Synthesis --> OutputValidator
    InsufficientEvidence --> OutputValidator
    Escalation --> OutputValidator : handoff envelope, no answer
    OutputValidator --> Synthesis : citations missing (1 retry)
    OutputValidator --> [*] : response emitted
```

Node budget note. C1/C2 = 1 LLM reasoning call + 1 synthesis call (fits the 2s SLO). C3 = 2 parallel evidence nodes + risk + synthesis. C4 = 3 parallel evidence nodes + risk + deep portfolio eval + synthesis (documented 6s budget).

---

5. Hybrid RAG + SQL Flow (the signature capability)

Worked example: *"Are any assets exceeding the allowed risk thresholds for our portfolio strategy?"*

```
sequenceDiagram
    autonumber
    actor RM as Risk Manager
    participant API as FastAPI
    participant G as Guardrail
    participant R as Router
    participant RET as Retrieval Agent
    participant PE as Portfolio Eval Agent
    participant EX as SQL Executor (read-only)
    participant RA as Risk Agent
    participant EE as Escalation Engine
    participant SY as Synthesis Agent
    participant OV as Output Validator

    RM->>API: query + JWT(role=risk_manager)
    API->>API: resolve allowed_portfolio_ids, trace_id
    API->>G: screen input
    G-->>API: clean (no override intent)
    API->>R: classify
    R-->>API: category=C3, risk=Medium,<br/>entities{threshold:"risk", scope:"all
portfolios"}

    par Policy evidence
```

```
        API->>RET: retrieve threshold rules
        RET->>RET: hybrid search over policy_chunks
        RET-->>API: doc_evidence[]<br/>[02_Risk_Management —Exposure Limits p.12]
<br/>[07_Risk_Framework —Approval Thresholds p.8]
    and Structured evidence
        API->>PE: fetch holdings + risk logs in scope
        PE->>EX: sql.risk_logs.by_score_threshold(min_score=70)
        EX->>EX: validate slots — RBAC filter — cap 200 rows
        EX-->>PE: rows(risk_id, asset, score, level, escalation_flag)
        PE->>EX: sql.holdings.by_portfolio_scope(ids=[...])
        EX-->>PE: rows(asset, sector, allocation_percentage, market_value)
        PE-->>API: sql_evidence[] (2 query_ids, row counts)
    end

    API->>RA: reconcile policy thresholds vs. observed scores
    RA->>RA: deterministic scorer (Low<30 / Medium 30–69 / High>=70)<br/>+ breach
detection vs. retrieved limits
    RA-->>API: risk_evidence[] — breached_thresholds[] — max_score=87 —
any_escalation_flag=TRUE

    API->>EE: evaluate E1–E8 on evidence
    EE-->>API: E2 fired (escalation_flag TRUE present)<br/>+ E6 (policy limit
breach)

    alt Escalation triggered
        API->>SY: (skipped)
        API->>OV: handoff envelope + evidence + reason
        OV-->>RM: ESCALATED — reviewer=Risk Committee —<br/>context: 3 assets >80 —
limits cited — no autonomous answer
    else No trigger
        API->>SY: synthesize from evidence bundle only
        SY-->>OV: answer + citations + contradiction notes
        OV->>OV: citations present? no advice? disclaimer?
        OV-->>RM: grounded answer with DOC + SQL citations
    end
```

What makes it "hybrid" rather than two systems

1. **Parallel fan-out** — policy and data are fetched concurrently, not sequentially.
2. **Reconciliation in the Risk agent** — the policy threshold (from a PDF) is compared numerically against the observed `risk_score` (from SQL). Neither source alone answers the question.
3. **Contradiction is a first-class output** — when policy and data disagree, the answer states both with both citations and, if a limit is breached, escalates.
4. **Dual citation** — every hybrid answer carries at least one `DOC:` and one `SQL:` citation.

---

6. Guardrail & Escalation Flow

```
flowchart TD
    Q["Analyst query"] --> P1{"Deterministic pattern match<br/>override — ignore — bypass — waive —<br/>skip assessment — despite escalation_flag —<br/>without approval — regardless of policy"}
    P1 -- match --> E1["E1 — Override / bypass intent"]
    P1 -- no match --> P2{"LLM intent classifier<br/>(paraphrased bypass?)"}
    P2 -- bypass --> E1
    P2 -- clean --> RTE["Router → agent path"]

    RTE --> EV["Evidence produced"]
    EV --> C2{"any escalation_flag = TRUE<br/>in scope?"} -- yes --> E2["E2 — Flagged asset"]
    EV --> C3{"risk_score > 80 or<br/>High with stale assessment?"} -- yes --> E3["E3 — Extreme risk score"]
    EV --> C4{"policy claim contradicts<br/>observed data?"} -- yes --> E4["E4 — Conflicting signals"]
    EV --> C5{"evidence missing or<br/>max score &amp;lt; RETRIEVAL_MIN_SCORE?"} -- yes --> E5["E5 — Insufficient evidence"]
    EV --> C6{"allocation / exposure exceeds<br/>a cited policy limit?"} -- yes --> E6["E6 — Regulatory limit breach"]
    EV --> C7{"approval_status Pending / Under Review<br/>and user asks to proceed?"} -- yes --> E7["E7 — Unapproved decision"]
    EV --> C8{"model confidence &amp;lt;<br/>CONFIDENCE_ESCALATION_FLOOR?"} -- yes --> E8["E8 — Low confidence"]

    C2 -- no --> OK
    C3 -- no --> OK
    C4 -- no --> OK
    C6 -- no --> OK
    C7 -- no --> OK
    C8 -- no --> OK["No trigger"]
    C5 -- no --> OK

    E1 --> REF["Refusal template<br/>(no method disclosed)"]
    E1 & E2 & E3 & E4 & E6 & E7 & E8 --> ESCN["Escalation node — TERMINAL<br/>build handoff envelope:<br/>trigger id — reason — evidence — reviewer role"]
    E5 --> INS["Insufficient-evidence template<br/>state what is missing"]

    OK --> SYN["Synthesis (evidence-only)"]
    SYN --> OV["Output validator"]
    REF --> OV
    INS --> OV
    ESCN --> OV
    OV --> RESP["Response envelope<br/>+ mandatory disclaimer"]

    style E1 fill:#ffdddd
    style E2 fill:#ffdddd
    style E3 fill:#ffdddd
```

```
    style E6 fill:#ffdddd
    style ESCN fill:#ff9999
```

Design invariant: escalation triggers are evaluated by **code, not prompts**. A prompt can be talked out of escalating; an `if` statement cannot. This is the only way Escalation Recall = 1.0 and Guardrail Effectiveness = 1.0 are achievable targets rather than aspirations.

---

7. Deployment View (local now → cloud later)

Same **image** and **schema** in every environment. Cloud move changes URLs, secrets, and orchestrator — not agents. Detail: `16_Production_and_Cloud_Readiness.md` + ADR-015…018.

```
flowchart TB
  subgraph LocalCI["Local / CI"]
    DC["docker compose"]
    APP1["finance-risk-api image<br/>uvicorn · APP_ENV=local|ci"]
    PG1[("Postgres: finance_auth_db + finance_risk_db/pgvector")]
    OTEL1["OTel collector optional"]
    DC --> APP1 & PG1 & OTEL1
  end

  subgraph Cloud["Cloud target (provider-agnostic)"]
    DNS["DNS + TLS + LB"]
    APP2["Same image · N replicas<br/>APP_ENV=staging|prod"]
    JOB["Jobs: migrate · ingest · eval"]
    PG2[("Auth DB")]
    PG2b[("Finance DB + vector")]
    OBJ["Object storage · policy PDFs"]
    SM["Secret manager → env"]
    OBS["Logs · OTLP traces · metrics"]
    DNS --> APP2
    APP2 --> PG2 & PG2b & SM & OBS
    JOB --> PG2b & OBJ
  end

  LLM["LLM provider OpenAI-compatible"]
  APP1 --> LLM
  APP2 --> LLM
```

Principle	Implication
Stateless app	Session memory in Postgres; scale by replica count; no sticky sessions
Two data stores	**`finance_auth_db`**: users/sessions · **`finance_risk_db`**: finance + `policy_chunks` (ADR-020)
Corpus portability	Ingest via `CorpusStore` (local FS → object storage)
Prod fail-fast	`APP_ENV=prod` rejects stub auth and placeholder secrets

Principle	Implication
Jobs ≠ runtime	Migrate/seed/ingest are CLI/jobs; container `CMD` is API only

8. Architectural Invariants (must hold in every sprint)

1. Synthesis has no tools; it composes only from the evidence bundle.
2. Only the Risk agent assigns a risk level.
3. No SQL reaches the database except through a named parameterized template executed by a SELECT-only role.
4. Escalation checks are deterministic and run after every evidence-producing node.
5. Every response carries at least one resolvable citation, or it is a refusal.
6. Agents communicate only through `GraphState` — no direct agent-to-agent messaging.
7. Every node emits a span; every request emits a structured log line with cost.
8. Every response carries the non-advice disclaimer.

> **Disclaimer.** Synthetic/educational data. Not investment advice.